

MEDASSIST-AI

Milestone-3 Report

DRUSHHTI KUHIKAR

Decision tree fine tuning and voting classifier

1. Project Description

MedAssist-AI: Medical Symptom Analysis and Disease Prediction System

MedAssist-AI is an Artificial Intelligence (AI) powered healthcare platform designed to assist users in analyzing symptoms and predicting possible diseases based on the symptoms provided. The system aims to support early identification of potential health conditions and provide AI-driven insights that can assist users in making informed healthcare decisions.

The platform includes multiple modules such as User Management, Symptom Analysis, Disease Prediction, Risk Assessment, Treatment Recommendation, Health Reports, and an Analytics Dashboard. Together, these modules contribute to the development of an intelligent healthcare decision-support system.

In the previous milestone, Decision Tree and Random Forest models were trained and evaluated for disease prediction using symptom-based data. The current milestone focuses on improving and extending the machine learning component of the system.

The main activities performed during this milestone included fine-tuning the Decision Tree model and developing a soft voting ensemble classifier. The voting approach combines the predictive capabilities of multiple machine learning models to generate a final prediction based on their combined probability estimates.

The implemented voting classifier combines the following models:

- Random Forest Classifier
- XGBoost Classifier
- LightGBM Classifier

The prediction probabilities generated by the three models are averaged using equal weights. The final disease prediction is obtained by selecting the class with the highest average probability. The developed voting classifier was evaluated using Accuracy, Precision, Recall, and F1-Score and was saved as a reusable model for future integration into the MedAssist-AI system.

2. Objectives

The objectives of this milestone were:

- To fine-tune the Decision Tree classifier by optimizing its hyperparameters.
- To evaluate and compare the performance of the baseline and fine-tuned Decision Tree models.
- To develop a soft voting classifier by combining multiple machine learning models.
- To combine the Random Forest, XGBoost, and LightGBM models using probability-based voting.
- To evaluate the performance of the voting classifier using Accuracy, Precision, Recall, and F1-Score.
- To save the trained voting classifier for future predictions and integration into the MedAssist-AI system.

3. Decision Tree Model Fine-Tuning

In the previous milestone, a baseline Decision Tree classifier was developed for disease prediction using the symptom-based dataset. In this milestone, the model was further improved through hyperparameter fine-tuning to analyse whether modifying the configuration of the Decision Tree could improve its predictive performance.

The fine-tuning process involved adjusting the model parameters and evaluating the resulting model using the testing dataset. The performance of the fine-tuned model was then compared with the baseline Decision Tree model using standard classification metrics, including Accuracy, Precision, Recall, and F1-Score.

The purpose of this process was to identify a suitable configuration for the Decision Tree model and determine the effect of hyperparameter optimization on disease prediction performance. Both the baseline and fine-tuned versions were analysed to understand the impact of tuning on the model's overall classification capability.

During the development of the ensemble model, it was also observed that the Decision Tree model had a mismatch in its disease class structure compared with the other selected models. Since probability-based voting requires compatible class outputs when combining prediction probabilities, including the Decision Tree model could result in inconsistent or unreliable ensemble predictions.

Therefore, although the Decision Tree fine-tuning experiment was completed and evaluated independently, the model was not included in the final voting classifier. The final ensemble

was developed using Random Forest, XGBoost, and LightGBM models, whose prediction outputs were compatible for probability-based soft voting.

```
# =====
# Hyperparameter Grid
# =====

param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [10, 20, 30, 40, 50, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

# =====
# Randomized Search
# =====

random_search_dt = RandomizedSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=20,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

random_search_dt.fit(X_train, y_train)
```

Fitting 3 folds for each of 20 candidates, totalling 60 fits

RandomizedSearchCV ⓘ ?

best_estimator_: DecisionTreeClassifier

DecisionTreeClassifier ?

```
# Best Hyperparameters
print("Best Parameters:")
print(random_search_dt.best_params_)

print("\nBest Cross Validation Accuracy:")
print(random_search_dt.best_score_)

Best Parameters:
{'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': None, 'criterion': 'entropy'}

Best Cross Validation Accuracy:
0.6982144573359356

# Best Decision Tree Model

best_dt = random_search_dt.best_estimator_

print(best_dt)

DecisionTreeClassifier(max_features='log2', random_state=42)
```

4. Voting Classifier Model Development

In this milestone, a soft voting classifier was developed to combine the predictive capabilities of multiple machine learning models. Ensemble learning was used to generate a final prediction by combining the probability outputs of the selected classifiers.

The models selected for the voting ensemble were:

- Random Forest Classifier
- XGBoost Classifier
- LightGBM Classifier

The previously trained models were loaded using their saved model files. Random Forest and LightGBM models were loaded using the Joblib library, while the XGBoost model was loaded from its saved JSON file.

Before creating the voting classifier, the dataset was prepared by separating the **diseases** column as the target variable and using the remaining 377 symptom-related features as input variables. The disease labels were encoded into numerical values using the LabelEncoder. The dataset was then divided into training and testing sets using an 80:20 split with a random state of 42.

The Decision Tree model was also loaded and evaluated during the model development process. However, it was not included in the final voting classifier because its disease class outputs were not compatible with the class structure of the other selected models. Since soft voting combines prediction probabilities class by class, mismatched disease classes could result in incompatible probability outputs and unreliable predictions.

Therefore, only the Random Forest, XGBoost, and LightGBM models were included in the final ensemble because their prediction probability outputs could be combined consistently.

For each test sample, prediction probabilities were generated using the three selected models:

- Random Forest → `predict_proba()`
- XGBoost → `predict_proba()`
- LightGBM → `predict_proba()`

The probability outputs were combined using equal weights. The average probability was calculated as:

Average Probability = (RF Probability + XGBoost Probability + LightGBM Probability) / 3

The final predicted disease class was selected based on the highest average probability. This approach allowed the ensemble to consider the predictions of all three models rather than relying on a single classifier.

A custom SoftVotingClassifier class was also created to encapsulate the three trained models and provide predict_proba() and predict() functionality. The completed voting classifier was saved using Joblib for future predictions and integration into the MedAssist-AI system.

```
import numpy as np

# Get prediction probabilities
rf_prob = rf_model.predict_proba(X_test)
xgb_prob = xgb_model.predict_proba(X_test)
lgb_prob = lgb_model.predict_proba(X_test)

# Average probabilities (equal weights)
avg_prob = (rf_prob + xgb_prob + lgb_prob) / 3

# Final prediction
voting_pred1 = np.argmax(avg_prob, axis=1)

print("✅ Voting predictions created successfully!")
```

✅ Voting predictions created successfully!

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, voting_pred1)
precision = precision_score(y_test, voting_pred1, average='weighted', zero_division=0)
recall = recall_score(y_test, voting_pred1, average='weighted', zero_division=0)
f1 = f1_score(y_test, voting_pred1, average='weighted', zero_division=0)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
```

Accuracy : 0.8295
Precision: 0.8792
Recall : 0.8295
F1 Score : 0.8476

5. Model Evaluation

The performance of the baseline and fine-tuned Decision Tree models, as well as the developed voting classifier, was evaluated using the testing dataset. The evaluation was carried out using four classification metrics: Accuracy, Precision, Recall, and F1-Score.

Weighted averaging was used for Precision, Recall, and F1-Score to account for the variation in the number of samples across different disease classes.

5.1 Decision Tree Fine-Tuning Evaluation

The performance of the baseline Decision Tree model was compared with the fine-tuned Decision Tree model to determine the impact of hyperparameter tuning.

Table 5.1: Comparison of Decision Tree Performance Before and After Fine-Tuning

Metric	Before Tuning	After Tuning
Accuracy	0.7549	0.7079
Precision	0.7600	0.7175
Recall	0.7549	0.7079
F1-Score	0.7543	0.7087

The results show that the fine-tuned Decision Tree model performed lower than the baseline model across all evaluation metrics. The baseline model achieved an accuracy of **0.7549**, while the fine-tuned model achieved an accuracy of **0.7079**. Similar reductions were observed in Precision, Recall, and F1-Score.

Therefore, the fine-tuning process did not improve the performance of the Decision Tree model. The baseline Decision Tree model was retained as the better-performing version for individual model comparison.

It is important to note that the Decision Tree was not included in the final voting classifier due to a mismatch in its disease class outputs compared with the other selected models. Since soft voting requires compatible class structures when combining prediction probabilities, including the Decision Tree could result in inconsistent probability alignment and unreliable predictions.

5.2 Voting Classifier Evaluation

The final voting classifier was developed by combining the following three models:

- Random Forest Classifier
- XGBoost Classifier
- LightGBM Classifier

A soft voting approach was used, where the prediction probabilities generated by the three models were averaged using equal weights. The disease class with the highest average probability was selected as the final prediction.

Table 5.2: Voting Classifier Performance

Metric	Score
Accuracy	0.8295
Precision	0.8792
Recall	0.8295
F1-Score	0.8476

The voting classifier achieved an **accuracy of 82.95%** and an **F1-Score of 84.76%**. The Precision score of **87.92%** indicates a high level of correctness in the predictions produced by the ensemble model.

These results demonstrate the effectiveness of combining Random Forest, XGBoost, and LightGBM through probability-based soft voting. By combining the prediction probabilities of multiple models, the ensemble classifier was able to provide strong overall performance for disease prediction.

The Decision Tree model was excluded from this ensemble because its disease class structure was mismatched with the class outputs of the other models. Therefore, its probability outputs could not be reliably aligned with those of Random Forest, XGBoost, and LightGBM for soft voting.

6. Conclusion

This milestone focused on fine-tuning the Decision Tree model and developing a soft voting classifier.

The fine-tuned Decision Tree did not improve performance compared to the baseline model. The baseline model achieved **75.49% accuracy**, while the fine-tuned model achieved **70.79% accuracy**. Therefore, the baseline Decision Tree remained the better-performing model.

The Decision Tree was not included in the final voting classifier because its disease class outputs were mismatched with those of the other models, making probability-based soft voting unreliable.

The final voting classifier combined **Random Forest, XGBoost, and LightGBM** using equal-weight probability averaging. It achieved **82.95% accuracy, 87.92% precision, 82.95% recall,** and an **84.76% F1-score**.

Overall, this milestone successfully implemented and evaluated a soft voting ensemble model for disease prediction.